

lytics teams are participating?

Team Name	
Lead(s)	
:-----	
----- :-----	
Product Data Insights	Carolyn Braza
Marketing Analytics	Jerome Ahye
HVS-FO	Max Fleisher
Sales Analytics	Melia Vilain & Noel Figuera
Customer Success Analytics	
Michael Arntz	
Digital Experience	
Dennis Charukulvanich	
People Analytics	Adrian Perez
Central Data Team	
Israel Weeks & Jong Lee	

Objectives of the FACE

- Efficiency: it is not uncommon for our teams to be asked similar questions (eg how are trials converting) but it is inefficient and duplicative for each team to tackle these questions on our own.
- Alignment: in cases where we are tackling similar questions, we need alignment of the assumptions and methodology we are using to answer those questions. The FACE ensures we all deliver cohesive data stories.
- Knowledge Share: this forum will give us a formal venue to learn from one another as opposed waiting for organic moments of knowledge sharing.

Outputs of the FACE

- Cadence: our teams will meet at least once every month if not more. We will knowledge share, align on joint quarterly projects, and develop/prioritize joint asks we have of the central data team. We will also have fun StaySpoke.
- Consolidated asks to central data team: we have an opportunity to streamline, consolidate, and prioritize our asks to the central data team. We also partner with the central data team on data program-level evaluations and decisions (eg BI tooling).
- Subject Matter Expert Lookup: we will develop a documented list of people and their associated areas of expertise. SME's will also be able to document their source of truth resources (eg snippets, dashboards, reports).
- Quarterly Projects: we will propose and pick cross-functional projects for us to work on quarterly. This will make us all more efficient by assigning 1 DRI with 2 or more code reviewers.
- Peer Review + Assumptions Approval: as a part of the quarterly projects, we will establish a code peer review and assumptions approval process that will ensure we will all be enthusiastic adopters of the ultimate output.
- Code Repo: after a project has gone through the peer review and assumptions approval process,

we will commit it to a repository that any data person can leverage in their work.

Examples: what are the types problems the FACE tackles?

- How do free and trial sign-ups convert?
- How do we link namespaces to Salesforce accounts?
- How do we link leads/INQ to Salesforce accounts?
- How do we identify business emails v. junk account emails?

Turn and Share Meetings

In line with our Collaboration value, Turn & Share sessions are a way for us to share our work with other analysts (or anyone at GitLab who is interested). These sessions began in October of 2023. More details on these sessions are outlined below:

Overall Goal: Encourage each of the analytic teams at GitLab to reach out to each other for data resources and ideas on how to solve reporting, statistical, and data science problems of various complexity levels.

General Guidelines for Discussions

1. Work does not need to be completed or in a finished state to be presented.
2. Slides are not always necessary, unless that is easier for the presenter to share their thoughts.
3. Active participation during sessions is what we're striving for. Questions at the time of presentation can be very helpful for all, especially the presenter.

Note: Goal is not to solve the problems we have in the meeting, rather to discuss and share ideas and thoughts. For deep discussions, follow-up meetings should be scheduled with interested Team Members.

Meeting Cadence

We strive to meet every three weeks, alternating between AMER and APAC time zones to facilitate consistent knowledge sharing and collaboration across functional analysts.

Sign Up for Presenters

We will collect sign ups and coordinate meeting schedules & facilitators in this sheet

Meeting Documentation

- Agenda
 - We collect notes, presentations, and Q&A during sessions.
- Recordings Meeting recordings are appended to the agenda post-session.

Presenting SAFE Data

At times, presentations will contain data that is considered SAFE. If this is the case it is important to take the following into considerations:

- Is everyone present able to see SAFE data? If the answer is no then it would be best to not share the visuals or create "mock data" that can be used for demonstration purposes.
- Is this being recorded? If the demo includes SAFE data, then the parts of the demo that include SAFE data should not be recorded. Even if everyone present can view SAFE data, visualized data cannot be recorded as the recordings are shared with a wider audience. It is best to stop the recording before sharing screen with the SAFE data in view or not record at all.

Working With Us

- Slack channel: functionalanalyticscenterofexcellence
- Open an issue in our Functional Analytics Center of Excellence project
- Join our biweekly meetings or monthly project turn and shares: ask in our Slack channel
- Read our meeting notes and watch meeting recordings [access required]

SECTION: enterprise-data/how-we-work/planning/index.md

Data Team Planning Process

The Data Team Planning Process is a pre-set activity that happens every quarter. The Planning Process adheres to GitLab's financial year/quarter schedule and Quarterly Objective and Key Results for prioritization and alignment. The goal of the Planning Process is to improve our ability to prioritize, plan and estimate work through collaboration and transparency.

This approach has many benefits, including:

1. It helps ensure the highest priority projects are being completed
1. It can help leadership identify issues that are blocked
1. It provides visibility into the work of the data team, including specialty analysts whose priorities are set from outside the data function
1. It encourages consistent throughput from team members
1. It makes clear to stakeholders where their ask is in priority
1. It makes clear to all team where their duty is in priority
1. It helps alleviate the pressure of planning the next iteration, as issues are already ranked

Quarterly OKR Planning

Data Team OKRs aspire to align with divisional and company OKRs as well as GitLab Yearlies. The Data Team will also create OKRs for Data Platform infrastructure development and these OKRs may not always map to immediate term Business Partner OKRs. Overall, OKRs constitute 50-60% of the Data Team's Quarterly Capacity, with Production Operations as the only established higher priority.

Currently we do our planning via a gSheet which gets communicated with all team members and stakeholders. Data Team OKRs are managed with GitLab Plan using a combination of Objectives, Key Results, Epics, and Issues.

This approach:

- enables async contributions and planning
- clearly defines Data Team priorities
- leverages KR health statuses for progress reporting

OKRs across the Data Team are written using GitLab's How to Write OKRs handbook page. The below formulas are used to write OKRs:

- Objectives: Verb + What you want to do + In order to/for/so that (what you hope to achieve or rationale for objective). Objective Example: Increase awareness of company in the market in order to increase sales.
- Key Results: Verb + what you're going to measure + from "x to y". Key Result Example: 100% of employees certified on OKR expectations and process.

Planning OKRs

OKRs are drafted in collaboration with our Business Partners and every KR that is put up for scheduling must have an opportunity canvas, see the project intake. Everyone at GitLab is welcome to open an opportunity canvas and we ask the Data Team members to contribute to the planning process by updating the gSheet with KRs they think are important and should be considered for scheduling. This collaborative approach ensures that all team members have a voice in the planning process and can contribute their insights and expertise.

Once the initial draft of OKRs is complete, the Data Leadership Team reviews and refines the OKRs to ensure alignment with company goals and strategic priorities. The finalized OKRs are then communicated to the entire Data Team and relevant stakeholders. If needed prioritization is raised and discussed within the Data Leadership Forum. KRs are converted into GitLab's OKR functionality.

Prioritization

The Data Team uses a prioritization framework to ensure that we focus on the most impactful work. Our prioritization process takes into account several factors:

1. Strategic Alignment: How well does the work align with GitLab's overall strategy and goals?
2. Time Criticality: How urgent is the work?
3. Revenue or Efficiency Impact: What is the potential financial impact of the work?
4. Regulatory or Legal Impact: Are there any compliance or legal requirements that necessitate this work?

We use a scoring system to evaluate each of these factors, which helps us calculate a priority score for each piece of the work. The priority score is calculated as follows:

$$\text{priorityscore} = (\text{Strategic Alignment} + \text{Time Criticality} + \text{Revenue or Efficiency Impact} + \text{Regulatory or Legal Impact}) / \text{Job Size}$$

Note: Carry-over KRs are prioritized (if still applicable) as we want to complete ongoing work first to achieve results.

Scoring breakdown

Value drivers:

Value	Weight	1	2	3
4	5			
-----	-----	-----	-----	-----
-----	-----			

Strategic Alignment	30%	Team	Department	Division
Multiple Divisions	Enterprise			
Time Criticality	40%	Nice to have	within the next year	within
the next 6 months	within next quarter		urgent ASAP	
Revenue or Efficiency Impact	20%	< \$5k	< \$50k	< \$500k
Regulatory or Legal Impact	10%	None/Nice to Have	Must Have	Fine
and/or Brand Damage <\$1M		Fine and/or Brand Damage <\$10M		Fine and/or Brand Damage > \$10M

Value	1	2	3	4	5	6
-----	--	-	-	-	--	---
T-Shirt Size	XS	S	M	L	XL	XXL

As the Job size is an important factor, we encourage to breakup KRs (over multiple quarters) which aligns with GitLabs value of iteration.

T-Shirt Sizing Approach

We use a T-Shirt sizing approach for quickly sizing the work required to deliver new issues, epics, and longer-term initiatives. The T-Shirt sizing approach is designed to support work breakdowns towards the creation of detailed work plans, but also intended to provide a sufficient level of detail to manage overall scope.

Size	Dedicated Person Time	Weight (issue points)	Examples
XS	1/2 Day	1	Update existing handbook page. data research/response. New Trusted Data Test. Opening AR to get access to a data source.
S	1 Day	2-3	New handbook page; typical triage issue. New dashboard on top of existing models. Align on data scope for new data source.
M	1 Week	5-8	New dashboard requiring new models. New data source with Stitch or Fivetran.
L	2-3 Weeks	13	New fact table implementation & testing. Full XMAU solution.
XL	1-2 Months	26	New Data Pump to new system. New Data Source with complex source API.
XXL	2-4 Months	52+	New Dimensional Model subject area with New Data Sources.

Assigning KRs

We encourage all Data Team Members to add their name to a KR in the list for the upcoming quarter (Column F - Data Team DRI) if they prefer to get a particular KR assigned to them. Possible reasons (not limited) to indicate you are up for a specific KR is that this is inline with your experience and expertise, previously worked on that area or the aspiration to learn more about it. You can also discuss with your manager the KRs you are interested in contributing to when you add your name as a Data Team DRI. It is not a guarantee it will also

be assigned as this depends on other factors as well.

Assigning KRs will happen towards the end of the quarter by adding team members and issue points Columns Q - X to the KRs.

Data Platform Vision and KRs

Although we encourage everyone within the team to contribute, in particular Staff+ level Data Engineers are expected to contribute to the long-term vision and strategic direction of the Data Platform, ensuring that our infrastructure and capabilities evolve to meet the changing needs of the business. They do this by discussing quarterly our Data Platform vision and translate if needed into KRs. Any updates or outcomes from these discussions are incorporated into the overall OKR planning process and communicated to the entire Data Team.

Key aspects include:

1. Identifying emerging technologies and trends in data engineering
1. Assessing current platform limitations and areas for improvement
1. Aligning platform development with overall company strategy
1. Proposing innovative solutions to enhance data processing, storage, and analysis capabilities
1. Collaborating with other teams to understand their evolving data needs

The resulting KRs from these discussions are integrated into the quarterly OKR planning process, ensuring that the Data Platform's development is aligned with both immediate business needs and long-term strategic goals.

OKR Tracking and Reporting

We use GitLab's built-in OKR tracking features to monitor progress throughout the quarter. Each Key Result is updated regularly, at least once per month, and the overall health status of Objectives is assessed based on the progress of their associated Key Results.

Progress on OKRs is reported in the following ways:

1. Weekly Data Team meetings: A brief update on key OKRs is provided.
2. Monthly in GitLabs OKR feature: A more detailed review of OKR progress, including any blockers or risks.

OKR Structure

mermaid

graph TD

```
Planning-gSheet[Planning Gsheet]
  Planning-iteration((Planning iterations))
    Objective[Objective]
      KeyResult1[KeyResult-1]
      KeyResultN[KeyResult-N]
    Epic[Epic]
      Issues1[Issues-1]
```

IssuesN[Issue-N]
Planning-gSheet-.->Planning-iteration-.->Planning-gSheet
Planning-gSheet-->Objective
Objective-->KeyResult1
Objective-->KeyResultN
KeyResult1-->Epic
Epic-->Issues1
Epic-->IssuesN

Work Breakdowns

Work breakdowns are always developed as a part of the Quarterly OKR Planning Process, but can also be leveraged to help scope and plan new initiatives, infrastructure projects, and similar multi-person or multi-week projects. The outcome of the work breakdown is a detailed description of the work to be performed, deliverables and responsibilities, and a high-level timeline.

- As a part of the Quarterly OKR Planning Process, work breakdowns are embedded in the KR Description.
- As a part of a stand-alone or ad-hoc initiative, work breakdowns are embedded in the appropriate Epic Description.
- As an example of a Work Breakdown, see this FY22-Q4 Data Platform Work Breakdown.

Work Breakdowns consider the following inputs:

1. Defined upcoming OKRs
2. OKR Reviews
3. New / forward looking insights
4. Team availability
5. Team member ambitions

The work breakdown is team effort and everyone is encouraged to contribute.

Twice-Weekly Iteration Planning

The data team works in two-week intervals, called iterations. Iterations start on Wednesdays and end on Tuesdays. This discourages last-minute merging on Fridays and allows the team to have iteration planning meetings at the top of the iteration.

Iteration planning should take into consideration:

- vacation timelines
- conference schedules
- team member availability
- team member work preferences (specialties are different from preferences)

The timeline for Iteration planning is as follows:

- Meeting Preparation - Responsible Party: Iteration Planner

- Investigate and flesh out open issues.
- Assign issues to the iteration based on alignment with the Team Roadmap.
- Note: Issues are not assigned to an individual at this stage, except where required.

Day	Current Iteration	Next Iteration
0 - 1st Wednesday	Iteration Start	
7 - 1st Tuesday	Midpoint	Any issues that are at risk of slipping from the iteration must be raised by the assignee
10 - 2nd Friday	The last day to submit MRs for review MRs must include documentation and testing to be ready to merge MRs are preferably not to be merged on Fridays, or on Thursday in the case of Family and Friends Day, (unless there is urgency, i.e. P1-Ops related MRs or in cases with tight timelines) to minimise impact on days where there is limited team member availability. Iteration is roughly final Iteration Planner verifies issue priority and team capacity for next iteration.	
13 - 2nd Monday	Last day of Iteration Ready MRs can be merged	
14 - 2nd Tuesday	Meeting Day All unfinished issues either need to be removed from iterations or rolled to the next Iteration Planning Sync-meeting to perform retro perspective on the current iteration and align/start on the next iteration according to the created iteration planning. All unfinished issues either need to be removed from iterations or will be automatically rolled to the next	

SECTION: enterprise-data/how-we-work/practical-guide/_index.md

Practical Guide

Our goal is to empower every team member to contribute to and benefit from our data ecosystem. This guide is designed to break down barriers and enable everyone, regardless of their role or technical background, to actively participate in our data projects.

By providing practical, step-by-step instructions, we aim to demystify the process of contributing to the Data Team's projects. Whether you're a seasoned data professional or someone from another department looking to leverage data for your work, this guide will help you navigate the technical aspects of making changes, resolving issues, and collaborating effectively.

We're committed to enabling teams to self-serve their data needs. This guide complements our

dbt change workflow, dbt guide, sql guide and ci jobs pages by focusing on the hands-on aspects of contribution.

Remember, every contribution, no matter how small, helps to enrich our collective knowledge and improve our data capabilities. We encourage you to dive in, experiment, and don't hesitate to ask questions in data. Together, we can build a more data-driven GitLab.

To make this guide more manageable and easier to navigate, we've broken it down into several focused pages, which you can find linked below:

- Working with the GitLab Web IDE
- CI Jobs You Should Know About
- Preparing Your MR for Merging

SECTION: enterprise-data/how-we-work/practical-guide/preparing-your-mr/_index.md

Steps to Getting Your MR Ready

To expedite the review process from an analytics engineer and eventual merging of your work, here are the steps you can take before tagging an analytics engineer for review. If you need help before completing these steps feel free to reach out sooner! It may take additional time for them to review your MR if additional support is needed from them.

Make sure the template is filled out completely

A top reason for slower reviews of an MR is an incomplete template. This page is designed to help you feel comfortable completing as much of the MR template as possible so that it can expedite the review process. You can read the section below, in Understanding the MR Template for more details about the MR template.

Build your changes

"Build Changes" is one of the CI jobs in the Pipeline. You can find more details about CI jobs to know about here. Before your MR gets merged, you will need to "Build Changes" and ensure it succeeds.

Grant yourself clones so you can test the data

Once your 'build changes' job succeeds, you can now grant yourself clones to the MR database which has been created. This allows you to test the changes of your MR. This step is important to save time and make sure we are not introducing inaccuracies to our database. Before you request to merge your new table into the database, you will likely have to grant yourself clones in order to test the changes.

You can view more information about how to grant yourself clones on the page about CI jobs here.

Because permissions are replicated from the production environment, if you are creating a net

new table you won't be able to access it using the "grant clones" CI job, you will need to ask the data engineers for access. You can view more instructions on how to do that in the grant clones section of the CI jobs page linked in the paragraph above.

Test the MR - either in SQL or in Tableau. Make sure it's doing what you think it is

Once you have access to your tables, make sure to test them! You can test them in Tableau, or Snowflake. Getting updates merged can be time consuming- not just from the review of the changes, but also just the time it takes to see your changes reflected once you update the database.

Even if you are making changes in a report table where nobody else will be negatively impacted if your code changes have a mistake in them, you are still holding up the process if you merge through changes that are incorrect and need to be changed again. When you are finished reviewing the changes from your MR and you leave your sign off that the changes are working as expected, you are committing to having tested your changes.

If you have already tested the changes in SQL/ Tableau before even creating the MR you can keep the testing to a minimum, but it is still a good idea to just double check that the SQL is working as you expected once you transferred it from Snowflake/Tableau to the MR.

Add a row count if you are updating an existing table

"Remote Testing" is where you can include the results of your remote testing. This could be a confirmation that you have tested these changes in Tableau. It could include a link to a spreadsheet with exported results of your testing - such as a pivot table comparing the results of the SQL code before and after the changes are made.

Before assigning this MR to a code owner for review, you should include a "Row Count Test". This is not one of the dbt tests listed in a previous step, instead it is a SQL query that you run yourself. This just counts the number of rows in the production version of the table, and the MR version. This only applies to MR's where you are editing an existing table and does not apply to newly created tables.

This row count catches things like when you accidentally type a join wrong, and it either dramatically increases or dramatically decreases the number of rows in the table. It is standard practice for almost every MR for a dbt model change. You can paste the resulting table right into this section of the template.

<details>

<summary>Sample SQL Code</summary>

SQL

```
SELECT
    COUNT() as rowcount,
    'prod' as source
FROM prod.yourschema.yourtablename
```

```
UNION ALL
```

```
SELECT
  COUNT() as rowcount,
  'MR' as source
FROM "your-mr-branch-name".yourschema.yourtablename
```

</details>

Please note: If your row count of your MR table is lower than what you see in prod, it may be because the MR database is outdated from prod. For example, if it is a table that updates daily and the MR is three days old, and the last time you built your changes was three days ago, then the tables will not have the same row count. You should be able to fix this by building the changes again, and granting yourself a new clone. You can ask an analytics engineer for help if you are stuck on this.

Understanding the MR Template

In this section we will break down the "Dbt Model Changes" template which is the template you should use for creating new report tables, or updating existing columns in our prep, mart, fact, dimension, or report layers.

Here is a breakdown of the MR template, one section at a time. The numbers on the image relate to text underneath the image which will explain that section. The numbers circled in red indicate sections which typically require action from you (the creator of the MR) for each MR, specifically for report tables.

1. "Closes" refers to the issue that this merge request will close. Typically in the development lifecycle, an issue will be created to describe work that needs to be done, and then the MR will be created to enact it. Once that MR merges, that issue should be ready to close.

- You may have some use cases that expand outside of this - for example you may create an issue for a change that needs to happen in Tableau, so when the MR gets merged the issue may not be ready to close, because you still have changes to make in Tableau. This is ok, just know that if you list the issue that the MR closes in this section of the template, it will automatically close when the MR gets merged.

- Creating an issue to accompany your MR if it does not yet have one could be helpful to the analytics engineer who you are partnering with. They can use the issue to discuss work, plan review meetings, and they can put weights on the issue and assign it to their schedule for the week so they can appropriately budget time for it.

2. In the scope section of the template, you need to describe the changes you are making in dbt and why you are making them. It does not need to be lengthy, but you want to use this section to help people who may not be familiar with the work you are doing to be able to quickly understand the "what" and the "why" so that it can be reviewed and merged.

- As a tip, you can use GitLab Duo's "Explain Changes" button if it is available to you on the MR. If not, you can copy and paste the changes tab of the MR and ask it to summarize the changes are being made. This may need some editing, but may help you get started.

3. In dbt, tests are predefined or custom checks configured in .yml files to validate the data in your models. These tests automatically check for specific conditions in the resulting tables, such as ensuring no null values, verifying unique rows, or maintaining referential integrity. This approach separates data validation from the model code, making it easier to ensure data quality and maintainability.

- You likely won't need to add tests to your tables when you're new to using dbt. The Analytics Engineer reviewing your MR can suggest tests if they think one is relevant. For more information about adding tests to your models, see the Trusted Data Framework handbook page.

4. Our CI jobs will build the models and environment included in the scope of your MR. You can leave this section as is, as the "Changes" tab will outline the MR's scope. In rare cases where additional models need to be built outside of those included in the buildchanges job, you can specify them here. If extra models need to be run, you can use the custominvocation job or the buildchanges job with a specified SELECTION.

5. Before an MR can be merged, you will need to "Build Changes", "Grant Clones" to yourself, and perform remote testing - either in Snowflake or Tableau.

6. In the "Verify" section you will verify that the changes you make are having the impact you expect. You will include the results below.

7. See the "row count" section above for information on how to add a row count. You should always do this as a best practice if you are updating an existing table. It may save you if you miss unexpected join errors.

8. You may need to take action on some of the items in the "Review" list. You can work with an analytics engineer on this section if you are not sure of the answers. The items that are most likely to need your action include:

1. Verifying that you have adequately tested the changes to ensure they do what you intended them to do.

2. You will need to run the required CI jobs in order to test the changes - make sure the changes all build successfully before requesting to merge the MR.

3. You should do your best to follow the team's style guide. The General section is particularly helpful. Once you tag an Analytics Engineer to review your MR, they can also help you with formatting the code according to the style guide.

4. If you are using VS Code, you can look into linting the code yourself which will provide some automatic formatting.

5. Before you merge an MR, you need to consider the downstream changes. For example, if you rename a column that is being used downstream, it may break those downstream tables in our data warehouse. If you rename a column that is being used in Tableau, any dashboards using that column will break (and it can be challenging to fix this error without reverting the changes).

Code owners and maintainers will review this section and check off the items as they complete their review

9. When you create an MR, you will likely start by assigning it to yourself. This helps you keep track of the MR's that you have open and in progress.

10. After completing testing and ensuring your changes are ready for review, assign the MR to an Analytics Engineer who is a code owner of the files you modified. Once they have reviewed and approved the changes, they will assign the MR to a maintainer for merging.

11. Labels can help you and your team keep track of your MR. If your team uses labels for your GitLab work, you can apply those same labels to your MR.

Assigning Reviewers

Once you have created your MR, tested the changes and are satisfied with them, filled out as much of the template as you can, and/or you need help from an analytics engineer it is time to assign a reviewer. Here is the official documentation about reviewers. Here is some information about the official review workflow.

If you typically work with an analytics engineer on your projects, it is your best bet to reach out to them to start with to ask who should review your MR. If you do not have an analytics engineer whom you typically work with, you can look in the 'reviewers' section of the MR (just under the pipeline) to see who owns the code of that table you are trying to modify.

You can choose any of the people listed on that list and use the right sidebar of the MR to assign them as a reviewer. You can also drop them a comment in the MR saying "Hi @reviewer, I have been working on this MR. I have tested the changes and filled out the template to the best of my ability, and was hoping you could take a look and let me know what we need to do to get this merged".

This is also a good place to look if you are blocked on one of the steps of creating the MR and are not sure who to ask for help.

Changes to Report Tables Versus Upstream Tables

When creating MRs as an analyst, it is helpful to understand that some MRs will be easier/quicker to review and push through than others. This depends on the complexity and location of the code changes.

If you have created a lengthy report table with a lot of transformations and custom logic, it will take analytics engineers longer to review these changes. Using CTE's instead of subqueries, and clearly labeling your code with comments and fully qualified column names can help someone who is unfamiliar with your code get up to speed more quickly.

Analytics engineers may also have feedback on your MR - they may catch inconsistencies, or have style recommendations. At GitLab we use a consistent style guide for how we format code. Analytics engineers may make suggestions to your code to help it meet our style guide - which makes it easier to use and maintain for everybody involved.

Report Tables Versus Mart/Fact Tables

There will be a different level of rigor expected of changes to mart/fact/dimension/prep tables than there will to report tables. The four types of tables listed previously are cross-functional, single-source-of-truth tables. These tables drive many different reports here at GitLab, and even the smallest change to these tables can have downstream impacts to many different reports and tables. Therefore, getting changes to these tables merged will require thorough testing and careful review, and be held to the highest standards of code cleanliness.

On the other hand, GitLab's data team wants to enable analysts to be able to create and maintain report tables, which will often contain data transformations that allow data to get into the right shape for reporting, such as a Tableau dashboard. Tableau dashboards will be more performant when they are pointing to a report table in Snowflake than when they are pointing to custom SQL.

If you are updating/creating a report table that is used in your dashboards, the review process will likely go more quickly than if you are updating a mart table which is used by many other tables/reports. When analytics engineers are reviewing report tables, they are looking to make sure:

- The code is readable
- The code works and does not break anything
- There are no redundancies
- There is no essential business logic

At GitLab, we avoid keeping business logic in our report tables that everyone else would benefit from being able to use upstream. Let's give a hypothetical example of this. Let's say you have a report table, and one of your columns is "Customer Size". In this column, you group your customers into revenue bands of <\$500, \$501-\$1000, and >\$1.000.

Now, let's say that your entire division generally wants to group customers in this way. Instead of creating this "Customer Size" CASE statement in every report table / Tableau report that reports on revenue, it would make sense to create that grouping upstream in the prep or fact table. Then, that column will flow through to every other table which is built from the base table.

This also helps tremendously when something happens like the company grows in size, and now every place where "Customer Size" is reported actually needs to group customers by the sizes of <\$1,000, \$1,001-\$5,000, or >\$5,000. These reasons together are why you may get feedback from an analytics engineer on your MR that one of your columns seems like business logic, and suggesting that it be moved upstream. A change like that is typically something that they would take over or assist you with.

Other Helpful Resources for Preparing MRs

Our DBT Guide: <https://handbook.gitlab.com/handbook/enterprise-data/platform/dbt-guide/>

Style Guide: <https://handbook.gitlab.com/handbook/enterprise-data/platform/dbt-guide/style-and-usage-guide>

General helpful information about using dbt: <https://handbook.gitlab.com/handbook/enterprise-data/platform/dbt-guide/general>

SECTION: enterprise-data/how-we-work/practical-guide/top-ci-jobs/_index.md

What is a CI Job (and Why Should You Care?)

Here is a complete list of all of our CI jobs at GitLab

<https://handbook.gitlab.com/handbook/enterprise-data/platform/ci-jobs/> . This section is a work in progress.

The two main jobs you will need to know when working in MRs is the Build Changes job, and the Grant Clones job. These will be used in almost every MR you submit!

Build Changes

To understand the build changes job, you need to understand what happens when you submit a merge request (MR). At GitLab, because we use our own product and have built out a suite of helpful CI/CD jobs in our pipelines, every time you submit an MR a 'new' database is created with your changes.

Initially, all that happens is that the code is copied over, and a blank database is created. The pipeline will scan your code for vulnerabilities, but it will not actually create any tables. However, you have the ability to ask GitLab to create a new merge request database - which specifically contains the code changes you are looking to merge.

For example, if you switch up the code in a table to add a new column and change how "Revenue" is calculated, you will likely want to test that code to make sure it is working as expected. You can do this in SQL, but what if you are changing a series of tables and they all rely on the changes to one another to test the new changes?

In your merge request database, GitLab can build the changes to every table which you have changed in your merge request (because you can make changes to more than one table at once), and any tables in between those tables. A "build" essentially means running the full query of the entire table and populating it with the full dataset.

Let's say you make a change to martcharges, and rptchargesweekly. But in between those two tables is the table martchargesweekly. The MR database will copy what is in production upstream of martcharges, and then it will build all three 'martcharges, martchargesweekly, and rptchargesweekly with your new changes - for example the new column you added, and the change to how revenue is calculated that you made in martcharges.

In order to trigger this "build" action, you can use the pipeline which is accessible right from the MR. Look for the pipeline (the following picture) and find the "Build Changes" job. All you need to do here is click the play button, and wait. It will then build the changes in the MR database to all of the tables which you have made changes to in your MR.

Grant Clones

Once you have built your changes, this means that there is a new database which you can access

that contains the MR's data which you have just created. Any code changes you have made are reflected, and any table which you have made changes to are available to query!

In order to access this, however, you need to grant yourself clones. If the table already exists (it is not a new table), then you can do this in the MR. You will find your pipeline, and use the "Grant Clones" job in the first step.

Unlike when you "build changes", you cannot just press the play button to the right of the name of the job. Instead, click on the green check mark that is the first in the line, and it will open all of the available jobs at that step. Look for "Grant Clones" and click on the text (not the play button).

This will bring you to a screen where you can easily add your key/value pair - which is how you tell the pipeline that you want it to grant you access to the MR database. Here you use GRANTTOROLE as the key. The value is your Snowflake role name, which is often your email. So if your name is John Smith, and your role is JSMITH, then you would put JSMITH as your value. You can grant access to yourself, or to others. You can only grant one clone at a time, even though there is the option to add more than one in the interface.

Once this finishes running, you can find the name of the branch you created for your MR in the list of databases which you have access to in Snowflake, and Tableau! This lets you test the impacts of your code changes.

Granting Clones to Net New Tables

If you are creating a net new table which has not existed yet, you will need to ask the Data Engineers to grant you access to the database/table. You can ask for permission for yourself and others, and you can ask for access to more than one table at once.

You need to specify the branch name, database, schema, and table. Most of the time you will be asking for access to tables in the PROD schema, so the general syntax is:
·branchnameprod·.schema.table

Here is an example message that you could copy and paste in the data-engineering Slack channel to ask for access to your table. If it is your first time and you are nervous to get it wrong, feel free to let them know in the message you are new and might make a mistake.

text

Hello @data-engineers, could you grant JSMITH and JDOE access to:

```
"2-create-report-tables-for-pushed-  
sqlPROD".restrictedsafecommonmartsales.rptpushedpipestartingfourthday  
"2-create-report-tables-for-pushed-  
sqlPROD".restrictedsafecommonmartsales.rptpushedpipestartingscaffold  
"2-create-report-tables-for-pushed-  
sqlPROD".restrictedsafecommonmartsales.rptpushedpipestartingdaily
```


Note: Before you ask the data engineers for access to your table, make sure you build the changes to your pipeline, and the build succeeds. If you have not yet built the changes (successfully), they will respond and tell you that they tried to grant you access, but that schema was empty. They won't be able to grant you access to the table until it exists (which happens when it builds successfully).

Custom Build Changes

Sometimes you may want to test MR changes that are not included in the scope of what gets built when you "Build Changes". This is most common when you make an adjustment to columns in an upstream table (such as a mart or prep table), but you actually want to test the impacts to those changes on your downstream report table.

If the report table does not have any changes involved in it, the MR database will not "build" that table, and you will not be able to query and test that table, by default. However, there is a way around this! You can set custom variables to your build changes job, or even use a similar job - custom invocation- to build your changes.

The easiest way to figure out how to do this is to use GitLab Duo. If you work for GitLab, GitLab Duo is available to you automatically when you are working in the GitLab WebIDE.

What you can do is go to this page of the handbook and copy the entire "Build Changes" and "Custom Invocation" sections. Then go back to your merge request, and open up GitLab Duo. Paste the entire contents of these jobs into the chat, hold shift and press enter to go to a new line, and write your prompt.

You can say, for example, "I have made a change to prod.restrictedsafecommonmartsales.martcrmopportunity in my MR. However, I want to test the impact that this would have on a table downstream - it is called prod.restrictedsafecommonmartsales.rptnewcharges. The normal build changes job did not build this table, because it is downstream and not involved in the MR directly. I want to build the mart table, and then also one table in between and then just my report table. How can I get this table to build?"

It will likely instruct you to go into the Build Changes job - so instead of clicking the play button as you did before, you click the actual text/ whitespace of the job, to the left of the play button. If you have not yet built your changes the screen for the custom variables will appear at this point, and you can input KEY: SELECTION ; VARIABLE: +prod.restrictedsafecommonmartsales.martcrmopportunity+prod.restrictedsafecommonmartsales.rptnewcharges.

If you have already run the build changes job, when you click on the build changes job in the pipeline, it will bring you to a page with a black box with colorful test detailing the job it has already run. You can specify your custom instructions by looking in the top right corner of the page, and finding the blue "reset" button. Click that, and select "Update CI/CD variables".

SECTION: enterprise-data/how-we-work/practical-guide/web-ide/_index.md

Introduction

The GitLab IDE is a powerful tool that allows us to write, test, and debug code directly within our GitLab instance. For the Data Team, this means we can efficiently work on our dbt models, and other data-related code without leaving the GitLab environment.

Before getting started, make sure you have created an issue using the appropriate template, most likely the "Standard Data Team Issue" template.

Create a new branch

Non-confidential issues

To create a branch and a merge request from a non-confidential issue, you can simply click the "Create merge request" button in your issue

While you have the option to create only the branch and not the merge request, and also to update your branch name and source by using the dropdown next to the 'Create merge request' button, it's best to create the merge request and branch simultaneously, using the default branch name and source (master).

Confidential issues

When working on a confidential issue, you may notice that you can't create an MR directly in the project. Instead, you're only presented with the option to create your branch and MR in a fork

However, you can still create a branch within the project for your work. To do this, click on 'Code' in the left sidebar, then select 'Branches'.

From there, click on the 'New branch' button to begin the process of creating your branch.

When creating your new branch, you'll need to consider the naming convention. Start the branch name with the issue ID (e.g., 21949-) to automatically link the future MR to the existing issue. Follow this with a descriptive name that closely aligns with the issue. For consistency, use snakecase or kebab-case (e.g., 21949-creating-a-branch-and-mr-from-confidential-issue). You'll also need to choose the source branch from which to create your new branch. In most cases, this will be the default branch 'master'.

width="600"/>

Once you've created your branch for this confidential issue, you'll usually be prompted to create a merge request on the next screen. If you don't see this prompt, don't worry. You can go to 'Merge requests' in the left sidebar, where you should see an option to 'Create merge request' for your recently created branch. Alternatively, you can click on 'New merge request' and choose your newly created branch as the source branch. Both methods will allow you to start the merge request process.

SECTION: enterprise-data/organization/_index.md

Data Team Organization

The Data Team Organization model is guided by five primary business needs:

1. The need for bespoke data solutions unique to the GitLab business.
1. The need for high-performance and reliable data storage and compute platform to support distributed analyst teams.
1. The need for centers of excellence for data technologies and advanced analytics.
1. The need for flexible data solutions driven by varying urgency and quality requirements.
1. The need to foster trust, compliance and value driven insights.

Based on these needs, the Data Team is organized in the following way:

1. Analytics Engineering: Transform raw data into clean, structured, and usable formats for data decision-making. The Lead Analytics Engineer serves as a stable counterpart for business departments and functional analytics teams.
1. Data Platform & Engineering Team: Center of Excellence for data technologies, including owning and operating the Data Stack
1. Data Science Team: Center of Excellence for advanced analytics, including delivery of data science projects to the business
1. Data Governance and Data Quality Team: Help build robust data governance practices and establish data quality frameworks for data quality monitoring and data quality improvement.

Data Team Operating Model

The Enterprise Data Team collaborates internally via Key Results. Key Results are planned on a quarterly basis and various team members from the four pillars of the Team can be assigned to a Key Result. The Key Result has a DRI who is the Directly Responsible Individual for the business outcome of the Key Result and leading the Team to success. Each respective pillar on the Team has flexibility to establish their own pillar specific ceremonies as well as processes on how they triage and assign P1-Ops and P3-Other issues that come up.

It is optional for team members to attend the ceremonies of other pillars. Team members are encouraged to attend other pillar ceremonies where attendance adds value.

At times, a pillar in the Enterprise Data Team may require collaboration and support from another pillar on an extended basis, lasting multiple quarters, and requiring consistent and deep support across P1, P2, and P3 issues. In these cases, a team member can be assigned as a stable counterpart to the pillar for an extended period of time with an end date established for the commitment where that team member will provide dedicated support across P1, P2, and P3 issues for the pillar. Expectations and capacity will be agreed at the start of the assignment and could change throughout in collaboration and agreement.

Below are the expectations of the DRI assigned to the Key Result:

1. Ensure completion of the opportunity canvas and ask for help when needed.
2. Schedule a work breakdown session with the Key Result Team. This can be either asynchronous or synchronous depending on the Key Result.
3. Schedule recurring stand-ups and working sessions as needed with the Key Result Team. This can be either asynchronous or synchronous and at a frequency that makes sense for the Key Result.
4. Provide a monthly update in the Key Result Issue in the OKR Project. Include percent complete and health status of the Key Result.
5. Raise risks and dependencies for successful completion of the Key Result with the data management team.

Analytics Engineering - Team and Stable Counterpart Assignments

Department / Division Functional Analytics Team Analytics Engineer Analytics Engineering Sub-Team			
-----	-----	-----	-----
Sales	Revenue Strategy and Analytics	@jkim @dantene	GTM
Marketing	Marketing Strategy and Analytics	@dantene	GTM
Finance	FP&A Analytics	@annapiaseczna	Finance
Customer Success	CS Strategy and Analytics	@mdrussell (Acting)	R&D
Product	Product Data Insights	@michellecooper	R&D
Engineering	Engineering Analytics	@michellecooper	R&D
Security	Engineering Analytics	@michellecooper	R&D
Support	N/A	TBD	TBD
People	People Analytics	@rakhireddy	People

Data Program Recruiting

Recruiting great people is critical to our success and we've invested much effort into making the process efficient. Here are some reference materials we use:

- Data Roles and Career Development to help existing team members and prospects understand growth opportunities
- a Take Home Test that we ask each candidate to complete; this test is good for the candidate and for us because it represents the type of work we perform regularly and if the candidate is not interested in this work it helps them make a more informed decision about their application

Data Roles and Career Development

Data Internships

See Data Team Internships.

Data Platform

- Data Engineering Job Family

mermaid

graph LR;

subgraph Data Engineering Roles

supe:jde(Junior Data Engineer)-->supe:de(Data Engineer);

supe:de(Data Engineer)-->supe:sde(Senior Data Engineer);

supe:sde(Senior Data Engineer)-->supe:fde(Staff Data Engineer);

supe:fde(Staff Data Engineer)-->supe:pde(Principal Data Engineer);

end

click supe:jde "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-engineer/junior-data-engineer";

click supe:de "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-engineer/";

click supe:sde "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-engineer/senior-data-engineer";

click supe:fde "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-engineer/staff-data-engineer";

click supe:pde "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-engineer/principal-data-engineer";

Intermediate and Senior Data Engineer Onboarding Timeline

| By Day 30 | By Day 60 | By Day 90 | By Day 120 |

| ----- | ----- | ----- | ----- |

| Complete People and Data Onboarding | Perform triage activities | Extract new data sources | Own a specific area of the data platform |

| Create a MR to contribute to handbook or templates | Investigate incidents and issues | Work on OKR assignments | Propose new ideas and come up with Data Platform improvement initiatives |

| Understand the current setup of the data platform | Make small/corrective changes to the platform infrastructure or data pipelines | Contribute on work breakdown | |

Data Analyst

- Data Analyst Job Family

mermaid

graph LR;

subgraph Data Analyst Roles

supe:ida(Data Analyst Intern)-->supe:jda(Junior Data Analyst);

```

    supe:jda(Junior Data Analyst)-->supe:da(Data Analyst);
    supe:da(Data Analyst)-->supe:sda(Senior Data Analyst);
    supe:sda(Senior Data Analyst)-->supe:fda(Staff Data Analyst);
end

```

```

    click supe:ida "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-analystdata-analyst-intern";
    click supe:jda "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-analystjunior-data-analyst";
    click supe:da "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-analystdata-analyst";
    click supe:sda "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-analystsenior-data-analyst";
    click supe:fda "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-analyststaff-data-analyst";

```

Intermediate and Senior Data Analyst Onboarding Timeline

By Day 30	By Day 60	By Day 90	By Day 120
Complete People and Data Onboarding	Extend an existing Tableau dashboard or complete the triage phase for a dbt issue	Run a project end-to-end as DRI	Create ERDs/Data Artifacts (e.g. dashboards) or complete a product evaluation
Complete First Issue: S to M T-Shirt Size			

Data Science

- Data Science Job Family

```

mermaid
graph LR;
    subgraph Data Science Roles
        supe:ds(Data Scientist)-->supe:sds(Senior Data Scientist)-->supe:stds(Staff Data Scientist)-->supe:pds(Principal Data Scientist);
    end
end

```

```

    click supe:ds "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-science/data-scientist-intermediate";
    click supe:sds "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-science/senior-data-scientist";
    click supe:stds "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-science/staff-data-scientist";
    click supe:pds "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-science/principal-data-scientist";

```

Intermediate and Senior Data Scientist Onboarding Timeline

| By Day 30 | By Day 60 | By Day 90 | By Day 120 |

| ----- | ----- |----- |----- |
 | Complete People and Data Onboarding | Meet stakeholders across the organization | Re-train or enhance an existing data science model | Make a contribution to improve the Data Science handbook, packages, or processes |
 | Start attending Data Science Team meetings | Refine/improve one data science dashboard | Work on OKR assignments | Take ownership of at least one quarterly OKR |
 | Understand the current data science systems and processes | | |

Analytics Engineering

- Analytics Engineering Job Family

mermaid

graph LR;

subgraph Analytics Engineer Roles

supe:aae(Associate Analytics Engineer)-->supe:ae(Analytics Engineer);

supe:ae(Analytics Engineer)-->supe:sae(Senior Analytics Engineer);

supe:sae(Senior Analytics Engineer)-->supe:fae(Staff Analytics Engineer);

supe:fae(Staff Analytics Engineer)-->supe:pae(Principal Analytics Engineer);

end

click supe:ae "https://handbook.gitlab.com/job-families/marketing/enterprise-data/analytics-engineer/associate-analytics-engineer";

click supe:ae "https://handbook.gitlab.com/job-families/marketing/enterprise-data/analytics-engineeranalytics-engineer-intermediate";

click supe:sae "https://handbook.gitlab.com/job-families/marketing/enterprise-data/analytics-engineersenior-analytics-engineer";

click supe:fae "https://handbook.gitlab.com/job-families/marketing/enterprise-data/analytics-engineerstaff-analytics-engineer";

click supe:pae "https://handbook.gitlab.com/job-families/marketing/enterprise-data/analytics-engineerprincipal-analytics-engineer";

Intermediate and Senior Analytics Engineer Onboarding Timeline

| By Day 30 | By Day 60 | By Day 90 | By Day 120 |

| ----- | ----- |----- |----- |

| Complete People and Data Onboarding | Extend an existing dbt Trusted Data Models | Run a project end-to-end as DRI | Create ERDs/Data Artifacts|

| Start attending Business Team synchronous meetings | Perform triage activities | | |

| Complete First Issue: S to M T-Shirt Size | | | |

Data Governance and Data Quality

Data Governance and Quality Analyst Job Family

- Data Governance and Quality Analyst Job Family

mermaid

graph LR;

```

subgraph Data Governance and Quality Analyst Roles
  supe:adgq(Associate Data Governance and Quality Analyst)-->supe:dgq(Senior Data Governance
and Quality Analyst);
  supe:dgq(Data Governance and Quality Analyst)-->supe:sdgq(Senior Data Governance and
Quality Analyst);
  supe:sdgq(Senior Data Governance and Quality Analyst)-->supe:sfdgq(Staff Data Governance
and Quality Analyst);
end

```

```

click supe:adgq "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-
governance-and-quality-analyst/data-governance-and-quality-analyst-associate";
click supe:dgq "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-
governance-and-quality-analyst/data-governance-and-quality-analyst-intermediate";
click supe:sdgq "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-
governance-and-quality-analyst/senior-data-governance-and-quality-analyst";
click supe:sfdgq "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-
governance-and-quality-analyst/staff-data-governance-and-quality-analyst";

```

Intermediate and Senior Data Governance and Quality Analyst Onboarding Timeline

```

| By Day 30 | By Day 60 | By Day 90 | By Day 120 |
| ----- | ----- |----- |----- |
| Complete People and Data Onboarding | Take up tasks related to assigned program | Own epic /
KR from planning to execution | Own specific data domain for data governance and data quality
improvement |
| Fully understand the data governance and data quality program, priorities and its strategy |
Investigate incidents and issues | Work on OKR assignments | Collaborate cross functionally and
identify areas for improvement |
| Create a MR to contribute to handbook or templates | | | |

```

Data Governance and Quality Program Manager Job Family

- Data Governance and Quality Analyst Job Family

```

mermaid
graph LR;
subgraph Data Governance and Quality Program Manager Roles
  supe:dgqp(Data Governance and Quality Program Manager)-->supe:sdgqp(Senior Data Governance
and Quality Program Manager);
  supe:sdgqp(Senior Data Governance and Quality Program Manager)-->supe:sfdgqp(Staff Data
Governance and Quality Program Manager);
end

```

```

click supe:dgq "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-
governance-and-quality-program-manager/data-governance-and-quality-program-manager";
click supe:sdgq "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-
governance-and-quality-program-manager/senior-data-governance-and-quality-program-manager";
click supe:sfdgq "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-
governance-and-quality-program-manager/staff-data-governance-and-quality-program-manager";

```


Data Management

- Data Management Job Family

mermaid

graph LR;

subgraph Data Management Roles

supe:md(Manager, Data)-->supe:smd(Senior Manager, Data);

supe:smd(Senior Manager, Data)-->supe:dd(Director, Data);

supe:dd(Director, Data)-->supe:sdd(Senior Director, Data);

end

click supe:md "https://handbook.gitlab.com/job-families/marketing/enterprise-data/manager-data/manager-data-intermediate";

click supe:smd "https://handbook.gitlab.com/job-families/marketing/enterprise-data/manager-data/manager-data-intermediate";

click supe:dd "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-and-insights-executive/director-data-and-insights";

click supe:sdd "https://handbook.gitlab.com/job-families/marketing/enterprise-data/data-and-insights-executive/senior-director-data-and-insights";

Data Manager Onboarding Timeline

| By Day 30 | By Day 60 | By Day 90 | By Day 120 |

| ----- | ----- | ----- | ----- |

| Complete People, Data, and Manager Onboarding | Meet everyone on the team and business data champions | Complete a Team Assessment | Draft a people development Roadmap |

| Understand the current setup of the data platform | Work on OKR assignments and map them to the data platform | Lead discussions with Users/Stakeholders on initiatives and OKRs | Draft a program development Roadmap (Process Improvements /Future State) |

| Add a new page to the handbook | Make regular contributions to the handbook spanning your area of management | Become DRI for major portions of the Data Handbook | System/Application Change Control Management of one or more modules |

Tool Technology Tandem

Tool Technology Tandems (TTT) are supporting to get the maximum value out of business opportunities we have in the Data Program. TTT are experts in a specific (software) tool or technology to support business opportunities or challenges we have by leveraging the tool or technology to the maximum. Although this is not the goal, we want to get the maximum value out of our technology stack. At the moment we see that we are not leveraging our technology stack to the maximum, where there are useful features or opportunities in our technology that could support in fulfilling business opportunities.

The reason is that from the technology side we don't know the business and from the business side we don't know the technology. The TTT will bridge this gap by understanding the needs and bring this together in a technological way. We expect from TTT to do consulting, guiding and educating.

Note: TTT will not search for business opportunities to use any tool feature. TTT has to understand business opportunities and translate this into what software could bring to the table.

A single TTT consists of minimum 2 and maximum 3 GitLab